Checkpoint 2

Due: October 19th, by 11:59pm

Overview

In the previous checkpoint you developed a detailed specification for a simple 2D, monochrome video game for the Atari ST. **It is critical that the specification is as thorough, complete and clear as possible before proceeding.**

In this checkpoint you will develop the models that will be used by the game. After developing and thoroughly testing your models, you will design, implement, and test a raster graphics library to support the required graphics capabilities of your game. Lastly, you will implement a rendering library that will render the current state of the model to the screen using the raster library.
It is important that you complete the section in the checkpoint in order as the order given in this checkpoint are based on what course material has been covered at the time. Each section should take approximately one week to implement and to create an appropriate and complete test module.

Testing

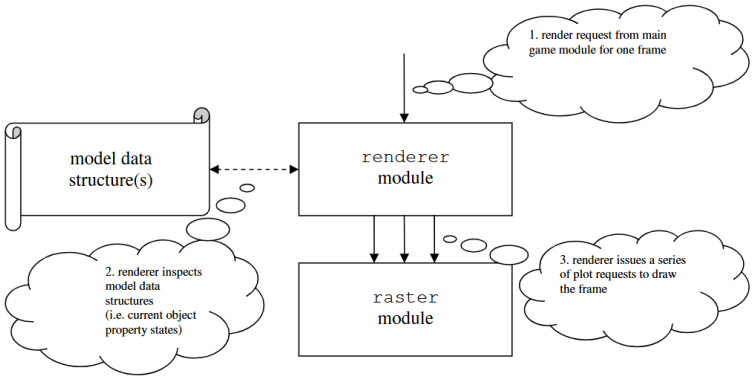
Making sure that all parts of the code in the game are adequately tested is important to ensure that when changes are made to code base that none of the existing functionality is broken when trying to add new functionality of fix a bug. On D2L there is a document on testing that must be used as a guide on what needs to be tested and how to build a test suite for a module. The test code for each module must test general cases and all edge cases to be considered a complete test module.

When writing tests, each module being tested will requires its own corresponding test module. For the game, this means that for checkpoint 3, there will be three separate test modules, one corresponding to each game module being developed.

Background: The Design of Video Games, Part 1

A video game is based on a *model* which describes how the game world is represented internally by the computer. The model captures the essential properties of the world's objects, their behaviours and interactions, the game's logic and the world's physics. In other words, it describes how the world's state is represented at any point in time, as well as how the world's state evolves over time. ***The model is independent of how a view of the world is presented to the user*** (e.g. as an image on the screen). The model has very little or nothing to do with bitmaps, plotting algorithms or other aspects of how the objects are ultimately displayed.

Rendering is the process of generating an image from a model[1]. In other words, the rendering module links the model module with the raster library. Later in this checkpoint will deal with rendering an individual frame of animation based on a static model state.



Model data structures are updated in response to events – perhaps asynchronous events triggered by key presses, synchronous events triggered by a clock tick, or conditional events triggered by either synchronous, asynchronous, or other conditional events. Checkpoint 3 will add event triggers and periodic render requests, so that the game world becomes animated[2].

THE MODEL

Overview

In this checkpoint the you will first design and implement the data structures for your game world's object types. At run time, objects will be represented by structures whose fields correspond to the objects' properties. Also, you will implement functions for manipulating objects according to their specified behaviours. Finally, you will implement event-handling functions for each of the specified asynchronous, synchronous and condition-based events.

Requirement: Model Data Structures

Object Type Models

Begin by designing the data structure(s) for representing the game world. You must identify each game world *object*[3] and object type, and decide on a representation for each of its properties. Ideally, the important details have already been described in your Checkpoint 1 specification.

[1] In 3D computer graphics, the term *rendering* has a more specific (but non-contradictory) meaning.

[2] Where the changes to the model are coming from are not shown on the above diagram as it is not relevant to the process of being able to render a complete frame.

[3] Although we are not using an object-oriented programming language, we will still think of the model as being comprised of various *objects*.

For example, in the specification a ball in Pong has position[4] as well as velocity[5] state. In C, this can be modelled as:

```
typedef struct {            /* type definition for Pong's ball object */
    unsigned int x, y;      /* position coordinates */
    int delta_x;            /* horiz. displacement per clock tick */
    int delta_y;            /* vert. displacement per clock tick */
} Ball;
```

Each object in your specification will be implement in a corresponding module. For example, the pong ball would be implemented in a module `ball.h` with code in the file `ball.c`.

Use Integer Value Where Possible

Try to stick to integer values whenever possible. When representing fractions, it is often preferable to use a pair of integers instead of a floating-point value.

Avoid decimal numbers at all costs! The Atari 68000 chip does not have a built in mechanism to handle floating point numbers, which means for a game that wants to use decimals a floating point library will need to be created.

Main Game Model

The main game will have a model associated with it, and it must have its definition in a file named `model.h` and any corresponding code in `model.c`.

If your game world has a large number of objects with heterogeneous types, collecting them by making intelligent use of arrays for multiple objects of homogenous type. For example:

```
typedef struct {
    Ball ball;
    Paddle paddles[2];      /* 0=left, 1=right */
    /* ... etc ... */
} Model;
```

Later, it will be possible to declare instance(s) of the model type(s). Initialize them according to the desired initial state of the game world. For example:

```
Model testPongSnapshot = {
    { 320, 200, -2, 2 },    /* the single Ball instance */
    {
        { ... },            /* left paddle instance */
        { ... }             /* right paddle instance */
    },
    /* ... etc ... */
};
```

[4]Note: the Atari ST's monochrome monitor resolution is 640 × 400. The ball position is therefore within this area.

[5]Time-relative properties such as velocity should be measured against a 1/70th of a second clock rate, as discussed elsewhere.

Requirement: Object Behaviour Functions

In the specification each object type has a set of behaviours. For each object add corresponding functions to the model for the object. For example:

```
#include "ball.h"

void move_ball(Ball *ball)
{
    ball->x += ball->delta_x;
    ball->y += ball->delta_y;

    /* ... what about collision detection? */
}
```

Also, declare ("prototype") these functions in `ball.h`, because these will need to be callable by other modules.

Requirement: Event Handler Functions

The last part of creating the models is to develop an `events` module (using the provided `events.h`). This module will consist of three (3) submodules, one for each type of event. The three module will be `asynchronous` with corresponding files `asynch.h` and `asynch.c`, `synchronous` with corresponding files `synch.h` and `synch.c`, and `conditional` with corresponding files `cond.h` and `cond.c`. For each of these modules you will develop *event_handler* functions which update the model's state for each event from the corresponding section of the game specification. The actual events won't be occurring yet, but their handlers can still be implemented and tested. These functions will delegate some or all of their tasks to the relevant `model` functions.

Requirement: Test Driver

Write one or more test driver programs which help verify your model implementation, including event handling. Since you have not yet implemented a method of displaying the model, instead, using text output (e.g. the C `printf` function) is far better right now.

Each test program should place the object or objects in desired position(s) and then use calls to the various event functions to verify that any expected changes to the model occur. After the initial placement and after each action the value of *every* object property should be output. For example in Pong the paddle could be placed on the screen and then use the up and down arrow key event functions to indicate on the next move*move* the paddle. Printing the paddle's x and y positions will show whether the event function is working correctly. For synchronous events (i.e. the clock) use repeated calls to the synchronous event function, with output to indicate that a tick occurred, to indicate a tick. In Pong place the ball at a given location with a given velocity, loop to correspond to the number of ticks to be tested, each time through the loop call the keypress the ball should move. Printing the ball's properties will show if the model's properties are correctly updated. If the ball was placed close to a wall with the correct velocity it can be determined if it bounces correctly.

Raster Graphics Library

The raster graphics library is not responsible for animation. Its routines are for plotting static images like pixels, lines, shapes and/or bitmaps to the frame buffer. The graphics will become animated in checkpoint 3.

Requirement: Provision of Low-Level Plotting Routines

The raster library is meant to be independent of your game, so you must to develop all of the following low-level routines, and perhaps others:

- clear_screen
- clear_region
- plot_pixel
- plot_horizontal line
- plot_vertical line
- plot_line (plot generic line between any two points)
- plot_rectangle
- plot_square
- plot_triangle
- plot_8bit_bitmap
- plot_16bit_bitmap
- plot_32bit_bitmap
- plot_character
- plot_string

These routines are “low level” from a design perspective because they will play a support role in rendering the game model to the screen, and elsewhere (i.e. other modules will invoke them to request fundamental screen drawing operations). They are also “low level” in the sense that they must not in turn rely on support from existing software modules. No operating system support is allowed in the implementation of your library, nor may it call third-party or standard C library functions. All plotting must be performed by your code, directly to the frame buffer.

Each function must take a *base* pointer as input, as well as function-specific input. The base pointer is the start address of the frame buffer into which the plotting must be performed.

Remember that your drawing routines must cope with screen boundary conditions. Forgetting about screen edges is a major cause of bugs.

For Full Marks

For all routines other than clear screen and plot pixel, if the image being drawn is requested to be drawn partially off the screen, then the routines must handle clipping, and if necessary for your game, wrap-around drawing of the raster.

Requirement: Game-Independence

Your raster graphics library must be independent of your game, in the sense that the library's design will not be coupled to game rules, object types, physics, events, etc.

For example, if the game was Pong, the raster graphics library would not *know* about paddles or balls, nor would it know about their properties or behaviours, ball movement or rebound physics, scoring rules, etc. If Pong's paddles and ball were each 4 bits wide, however, the library could include a *generic* routine for plotting *arbitrary* 4-pixel-wide bitmaps. This routine could later be called variously by other code to accomplish the plotting of paddles, the ball, and other 4-pixel wide images used in the game.

In theory, it should be possible to reuse the library for other games.

A Note On Text

Games often require the printing of text. This is easily accomplished using "bitmap" fonts: a bitmap (called a "glyph") is provided for each character that might be plotted. Therefore, as long as bitmaps can be plotted, it will later be possible to print text in a straightforward way.

It is possible to design your own bitmap fonts (or maybe to find one that is free to use). Alternatively, it is possible to use the Atari ST's own bitmap font table. Your instructor will discuss how to accomplish this later. As well, `I:\Labs\CompSci\Resources\2659\project\fonts` folder contains a number of 8×16 font tables that you can use. For future reference, the font table is an array of 256 8×16 glyphs, indexed by ASCII value. If you plan to use these, then consider including a plotting routine for 8-bit-wide bitmaps as part of implementing the raster library for this checkpoint.

A Note on Smart Graphical Layout for Convenience and Efficiency

As was described in an earlier document, it is easiest (for architectural reasons) to base the game's graphics as much as possible on bitmaps which have widths of 8, 16 or 32 pixels (or multiples thereof) – ideally a suitable set of bitmaps have already been developed as part of your Checkpoint 1 game specification. If possible, it is convenient to arrange the frame buffer area so that the plotting of horizontal lines, bitmaps, etc. is aligned as much as possible on byte or word boundaries. When possible, this can lead to faster plotting and sometimes to simpler plotting algorithms.

The above are not possible in every game. For example, if some objects move smoothly and horizontally across the screen, then plotting (at least of those objects) will not always be byte-aligned.

Raster Test Driver

Your raster graphics library must be accompanied by a *test driver* program which invokes and thoroughly exercises each of its plotting routines.

Add tests for each function you write as you go. If you avoid prompt testing, it will be impossible to build robust code. Your tests should include testing a variety of scenarios and all boundary conditions.

A Note On Efficiency

At first, focus on the correctness of the plotting routines. Implement them in C, and avoid tricky optimizations.

Fast and Efficient Raster Algorithms

To decrease drawing speed, which will decrease rendering speed, and improve game play, all raster code should replace all multiplication, division, and modulus operations with other operations that require less processing time.

Once a routine is working correctly, it may then be possible to speed it up (sometimes by a large factor). Having a thorough set of tests will allow you to verify changes quickly and repeatedly.

Once a routine is working as quickly as possible in C, rewrite the routine in assembly language. **Note:** It is not required that the plotting routines implemented in C before coding them in assembly, but that might make it easier.

For Partial Marks

An implementation of the raster library that is not completed written in assembly will not be eligible for full marks.

Skeleton Code

Your graphics library must be implemented as a C module named *raster* consisting of the following three files:

- raster.h ←header file describing the module's public interface, as needed by clients
- raster.c ←contains definitions for routines implemented in C (i.e. most of them)

- rast_asm.s ←contains definitions for routines implemented in assembly (if any)

To assist you, your instructor has placed a skeleton project in:

```
I:\Labs\CompSci\Resources\2659\project\stage2\skeleton
```

Also included in this folder are a make file for the project and a test driver skeleton.

A Note on the Proper Use of Header Files

A header file is intended to contain declarations relevant to a corresponding project module. Therefore, header files should contain information needed by client modules, including public function prototypes and public type definitions. Header files must *not* contain variable declarations or function definitions which result in space being reserved or in object code generation! Among other problems, this can lead to link errors (what if the same `.h` file is included in more than one `.c` module?). For some reason, a common student mistake is to define arrays for bitmaps in header files. Avoid this! Types, constants, functions, etc. which are purely for internal use in the module should similarly not be mentioned in the header.

RENDERER

Overview

In this stage, you will write a module which links the model with the low-level graphics library, so that it is possible to render an individual, static frame of animation based on a static snapshot of the world.

Before starting on this stage it is critical that the model's data structures are complete, well designed and implemented, and also that the raster module contains a reliable plotting routine for each type of graphics primitive (e.g. bitmap, shape, line, etc.) required by your game.

Once this checkpoint is complete, Checkpoint 3 will involve creating a main game module which updates the model based on user input and clock tick events, and that repeatedly renders frames of animation to achieve an animated, evolving game world.

Background: The Design of Video Games, Part 1 Recap

Recall that *rendering* is the process of generating an image from a model.
Note that the main game module (which will be written in the next checkpoint) will eventually invoke the renderer module once for each frame of animation. In this stage, we are concerned only with rendering a single frame (i.e. a static, full-screen image) based on a static model (i.e. a snapshot of the model data structure(s) at a single point in time).

Requirement: Renderer Module

Provide a *render*_object function for each type of object which can appear on-screen. For example, Pong would include *render_ball* and *render_paddle* functions.

In addition, provide a master *render* function which renders a whole frame based on the current state of the model. Of course, it delegates to the various *render_object* sub-functions.

Each of these functions must take a frame buffer *base* pointer as input, as well as function-specific input. The base pointer points to the buffer into which the plotting must be performed. The function-specific input is the relevant portion of the model data structure(s).

For example:

```
void render(const Model *model, UINT8 *base);
void render_ball(const *Ball, UINT8 *base);
/* ... etc ... */
```

For Full Marks

For all object specific render functions (e.g. render_ball in Pong) must have as the parameter the object being rendered and **not** the overall game model. The master render function is the only render function that can have the game model as a parameter.

Requirement: Test Driver

The test driver for the rendering module is pretty simple. ***Make a copy of your model test driver***, and replace the printed output with calls to the master render function. If your model test driver is complete this will allow you to test the rendering code with a full set of different static model configurations. After each call to the master render function add code so that the tests wait for a keypress, and then clear the screen.

Minimizing Screen Plotting

In the first version of your renderer, it is acceptable to plot an entire frame each time the master render routine is invoked. In Checkpoint 3, an easy way to animate the game is to clear the screen and then completely re-render the entire game model, for each frame. Of course, the disadvantage of this approach is that it is slow. It is therefore difficult to achieve, smooth, high-quality graphics with a high frame rate.

Once basic rendering is working, you may wish to consider improving the renderer so that it only re-plots the aspects of the model that have changed since a previous invocation. There is more than one way to accomplish this – working out the details is your responsibility.

Minimizing screen plotting is not mandatory before starting Checkpoint 3; however, it is strongly recommended that you start thinking about how to do it. By the end of Checkpoint 3,

it is likely that your game's graphics will suffer very noticeably if it naively and inefficiently over-plots.

Background: Plotting Text using TOS's Bitmapped Font Table

For the plotting of text, it is possible to define your own "glyph" bitmaps – one for each character that may need to be plotted. These bitmaps can then be collected into an array of glyphs indexed by ASCII value. Assuming appropriate bitmap plotting routines exist, plotting strings of text is straightforward.

Alternatively, the Atari ST's operating system, TOS, has a bitmapped font table that you can use if you like. The font table is made up of 256 8×16 bitmaps – one bitmap per character in the extended ASCII character set. The following code shows how to access the table:

```
#include <linea.h>
#include <osbind.h>

...

UINT8 *base = Physbase();
UINT8 *fontTable;
char ch = 'a';    /* example character to plot */

linea0();    /* required TOS system call */
fontTable = (UINT8 *)V_FNT_AD; /* start address of font table */
```

Treat `font` as an array of bytes. Indexing it at the ASCII value of `ch` will give a byte. This byte is the first row of `ch`'s bitmap! *The second row is 256 bytes further down the table*, and so on...

```
*base = *(fontTable + ch);
*(base + 80) = *(fontTable + ch + 256);
*(base + 160) = *(fontTable + ch + 512);
...
```

For example, the following code plots a in the upper-left of the screen:
Of course, this can be accomplished more compactly using a 16-iteration loop.

Depending on how you have coded your bitmap plotting routines, it is likely that you expect bitmap rows to be contiguous, not 256 bytes apart! If you would rather that glyph rows be contiguous, write a program that *rips* the TOS font table and produces a modified font table with the glyphs stored as desired. In fact, this will be mandatory eventually, since a later checkpoint will eliminate reliance on TOS altogether. If you are printing strings, It is actually more convenient to leave the font table in its original form.

Other Requirements

Your code must be highly readable and self-documenting, including proper indentation and spacing, descriptive variable and function names, etc. No one function should be longer than approximately 25-30 lines of code – decompose as necessary. Code which is unnecessarily complex or hard to read will be severely penalized.

For each function you develop, write a short header block comment which specifies:

1. Its purpose, from the caller's perspective (if not perfectly clear from the name);
2. The purpose of each input parameter (if not perfectly clear from the name);
3. The purpose of each output parameter and return value (if not perfectly clear from the name);
4. Any assumptions, limitations or known bugs.

At the top of each source file, write a short block comment which summarizes the common purpose of the data structures and functions in the file. This should also include a normal ID header, but should also include team member names and the author's name.

You should add inline comments if you need to explain an algorithm or clarify a particularly tricky block of code, but keep this to a minimum.